MODULE 2

PROJECT REPORT

PROJECT

Analysis of Real-Time Object Detection Using YOLO

STUDENT DETAILS

Name	Subham Roy
Section	C
Roll No.	15
Enrollment No.	12023002001218
Program	B.Tech CSE

Certificate / Approval

This is to certify that the project titled “Analysis of Real-Time Object Detection Using YOLO” has been prepared as part of Module 2 academic work under the guidance of the concerned faculty.

Project identification fields are completed below for submission.

Field	Details
Student name	Subham Roy
Registration / Roll No.	12023002001218 / 15
Department	Computer Science and Engineering
Faculty / Guide	Concerned Faculty
Date	29 September 2026

Declaration

I declare that the implementation and documentation submitted for this project are prepared for academic purposes. Third-party software libraries, pretrained model checkpoints, and datasets remain subject to their respective licenses and terms.

Student Signature: Subham Roy Date: 29 September 2026

Acknowledgement

I acknowledge the guidance provided by the faculty and the open-source computer-vision ecosystem that makes reproducible object-detection experimentation possible. The implementation has been structured so that the runtime configuration and measured outputs can be recorded transparently.

Abstract

This project implements a real-time object detection and performance-analysis pipeline using YOLO and OpenCV. The system accepts image, video, or webcam input, performs frame-level inference, visualizes detected bounding boxes and class confidence, and records runtime measurements including inference latency, frame rate, detection count, mean confidence, and the 95th-percentile inference latency. The design separates input handling, model inference, metric collection, visualization, and reporting so that each stage can be tested and reused. The accompanying experimental protocol evaluates how image size and confidence threshold influence runtime behavior. The project is intended as an academic demonstration of real-time visual perception rather than a safety-certified autonomous-driving system.

Table of Contents

1. Introduction

2. Problem Statement

3. Objectives

4. Scope

5. System Requirements

6. System Architecture

7. Methodology

8. Implementation Details

9. Performance Metrics

10. Experimental Protocol

11. Expected Output and Evidence

12. Results Recording

13. Limitations

14. Future Scope

15. Conclusion

References

Appendix A - Run Commands

Appendix B - Project Structure

1. Introduction

Module 2 Option 5 specifies a project on the analysis of real-time object detection using YOLO. The implementation extends the core detection task with a measurable performance layer, making it possible to study latency, frame rate, and detection behavior under controlled configurations.

The system is designed around a common perception loop: acquire a frame, run detection, obtain bounding boxes/classes/confidences, visualize the result, and record the measurements needed for later analysis.

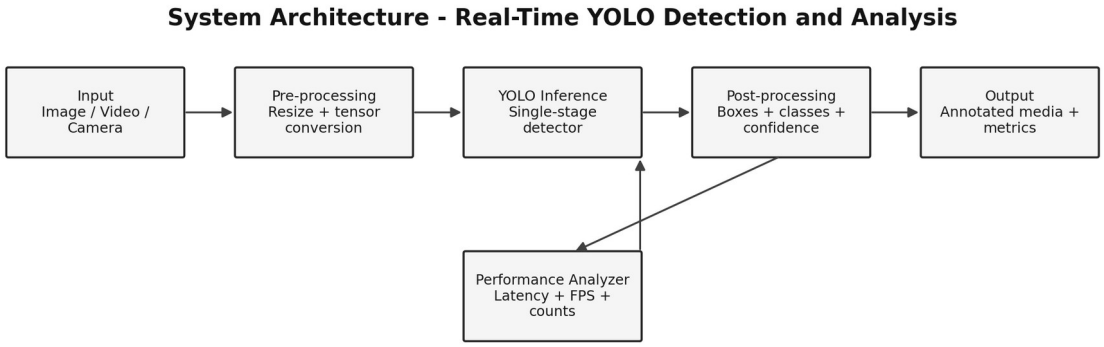


Figure 1. High-level system architecture of the project.

2. Problem Statement

A real-time object detector must produce useful detections while processing a continuous stream of frames. Merely demonstrating bounding boxes does not reveal whether the system can sustain an acceptable processing rate. The project therefore addresses two linked questions: (1) can YOLO identify objects in incoming visual data, and (2) what runtime behavior does the detector exhibit under the selected configuration and hardware?

3. Objectives

- Implement YOLO-based object detection for image, video, and webcam sources.
- Visualize detected objects with bounding boxes, class labels, and confidence scores.
- Measure per-frame inference latency and frame rate.
- Record detection counts and confidence statistics for analysis.
- Generate machine-readable CSV/JSON results and grayscale plots.
- Provide a controlled experiment protocol for comparing configurations.

4. Scope

- Pretrained YOLO inference; custom model training is outside the core submission scope.
- Single-camera visual input through images, video files, or a webcam.
- Performance analysis on the local computer used for the experiment.

- Academic visualization and reproducibility; not a safety-certified autonomous-vehicle stack.

5. System Requirements

Component	Requirement / Role
Language	Python 3.8+ recommended by the current Ultralytics installation documentation.
Detection framework	Ultralytics YOLO Python package.
Vision I/O	OpenCV.
Numerical / reporting	NumPy, Matplotlib, Pandas (for optional analysis workflows).
Model checkpoint	Configurable pretrained YOLO checkpoint; default project setting: yolo26n.pt.
Hardware	CPU works for inference; a compatible GPU can accelerate execution.
Input	Image, video file, or webcam index.

The PDF specifies Python as an allowed language/tooling choice. The current Ultralytics documentation provides Python and CLI prediction workflows and currently illustrates yolo26n.pt in its quickstart.

6. System Architecture

The architecture is intentionally modular. Input handling isolates OpenCV capture. The detector wrapper isolates the Ultralytics API. The metric tracker records timing and detection statistics, while the reporting module converts those measurements into CSV, JSON, and plots.

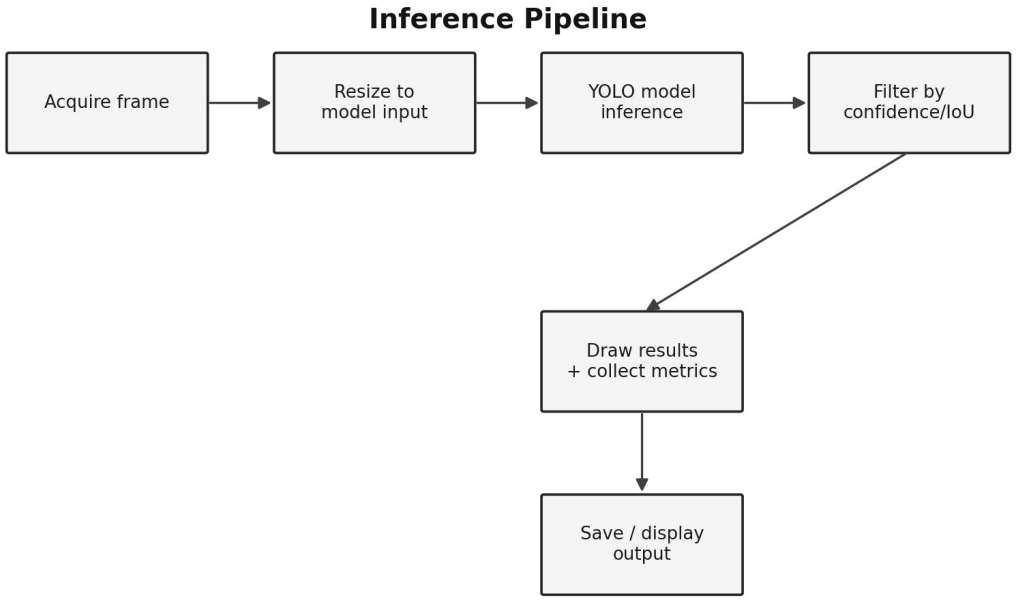


Figure 2. Frame-level inference pipeline.

7. Methodology

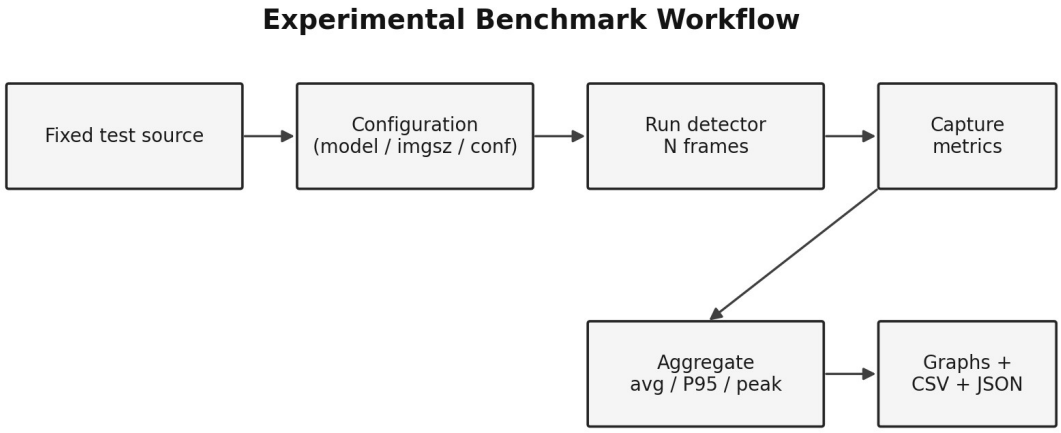


Figure 3. Controlled benchmark workflow.

The benchmark procedure keeps the input source fixed and varies one configuration factor at a time. Suggested experiments use image sizes of 640, 512, and 320 pixels, together with confidence thresholds such as 0.25 and 0.50. The same checkpoint, input sequence, frame count, and computer should be used across runs.

8. Implementation Details

8.1 Detector wrapper

`src/detector.py` loads the selected YOLO checkpoint, resolves the execution device, calls model prediction, renders the returned result, and converts model outputs into a summary containing total detections, per-class counts, and mean confidence.

8.2 Input and output handling

`src/io_utils.py` interprets numeric sources as webcam indices, creates output directories, and initializes a video writer using the source frame dimensions and frame rate when available.

8.3 Metrics layer

`src/metrics.py` uses a monotonic high-resolution timer to capture per-frame loop time and model inference time. It also derives frame rate after the first processed frame and calculates aggregate statistics including mean and P95 inference latency.

8.4 Reporting layer

`src/reporting.py` writes `metrics.csv`, `summary.json`, and three neutral-tone graphs covering FPS, inference latency, and detections per frame. These artifacts create an evidence trail for the report.

9. Performance Metrics

Metric	Definition	Why it matters
Inference latency	Elapsed time for one detector inference call.	Measures computational cost per frame.
Average FPS	Mean frame rate after the first frame.	Indicates sustained processing throughput.
Peak FPS	Maximum observed frame rate.	Shows best observed throughput but is sensitive to short bursts.
P95 latency	95th percentile inference time.	Highlights slower tail frames that an average can hide.
Mean confidence	Average confidence over detections.	Summarizes confidence of retained predictions.
Detections/frame	Number of boxes produced in a frame.	Shows scene/object density over time.

10. Experimental Protocol

Each experiment should process a fixed short traffic or general-purpose video. A 300-frame cap is recommended for repeatability. Record the computer hardware, OS, Python version, model checkpoint, image size, confidence threshold, IoU threshold, device, and frame count.

Run	Image size	Confidence	Purpose
A	640	0.25	Baseline
B	512	0.25	Lower input size
C	320	0.25	Speed-oriented test
D	640	0.50	Higher confidence threshold

The experiment table is a protocol, not a claim about measured performance. Actual values must be obtained by running the project on the target computer.

11. Expected Output and Evidence

- `annotated_output.mp4` or `annotated_image.jpg` showing detected objects.
- `metrics.csv` containing one row per processed frame.
- `summary.json` containing aggregate runtime statistics.
- `plots/fps_over_frames.png` showing frame-rate behavior.
- `plots/inference_latency.png` showing per-frame inference latency.
- `plots/detections_per_frame.png` showing scene detection density.

12. Results Recording

To preserve academic integrity, this report deliberately does not invent performance values. After running the project on the submission computer, copy the values from each `summary.json` into the final results table and attach the generated graphs and an annotated frame/video screenshot. This keeps the report aligned with the actual experiment.

Run	Avg FPS	Avg latency (ms)	P95 latency (ms)	Mean confidence	Frames
A - Baseline	_____	_____	_____	_____	_____
B - 512 input	_____	_____	_____	_____	_____
C - 320 input	_____	_____	_____	_____	_____
D - conf 0.50	_____	_____	_____	_____	_____

13. Limitations

- Detection quality depends on the pretrained checkpoint, scene conditions, and object scale.
- Runtime varies significantly with CPU/GPU hardware and software configuration.
- A pretrained generic detector is not equivalent to a safety-certified automotive perception system.
- The project does not implement sensor fusion, lane estimation, tracking, planning, or vehicle actuation.

14. Future Scope

- Train or fine-tune a checkpoint on a domain-specific traffic dataset.
- Add multi-object tracking and trajectory estimation.
- Export to an optimized runtime such as ONNX/TensorRT when appropriate.
- Add vehicle-focused datasets and evaluate mAP/precision/recall alongside runtime metrics.
- Integrate detection output with a broader autonomous-vehicle perception stack.

15. Conclusion

The completed project design provides an end-to-end software pipeline for real-time YOLO detection and measurable performance analysis. Its main academic value is the combination of practical object detection with reproducible runtime evaluation. The separation of detector, metrics, I/O, and reporting components also makes the system easier to test, explain, and extend.

References

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” arXiv:1506.02640, 2015; published at CVPR 2016. https://arxiv.org/abs/1506.02640

[2] W. Liu et al., “SSD: Single Shot MultiBox Detector,” arXiv:1512.02325, 2015; published at ECCV 2016. https://arxiv.org/abs/1512.02325

[3] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,” arXiv:1506.01497, 2015; published at NeurIPS 2015. https://arxiv.org/abs/1506.01497

[4] Ultralytics, “Install Ultralytics,” and “Modes / Predict,” official documentation, accessed 28 Sep 2026. https://docs.ultralytics.com/quickstart and https://docs.ultralytics.com/modes

Appendix A - Run Commands

python -m src.main --source path/to/image.jpg --model yolo26n.pt --run-name image_demo

python -m src.main --source path/to/video.mp4 --model yolo26n.pt --run-name video_demo

python -m src.main --source 0 --model yolo26n.pt --run-name webcam_demo

python -m src.main --source path/to/video.mp4 --model yolo26n.pt --run-name benchmark_01 --no-display --max-frames 300

python -m pytest -q

Appendix B - Project Structure

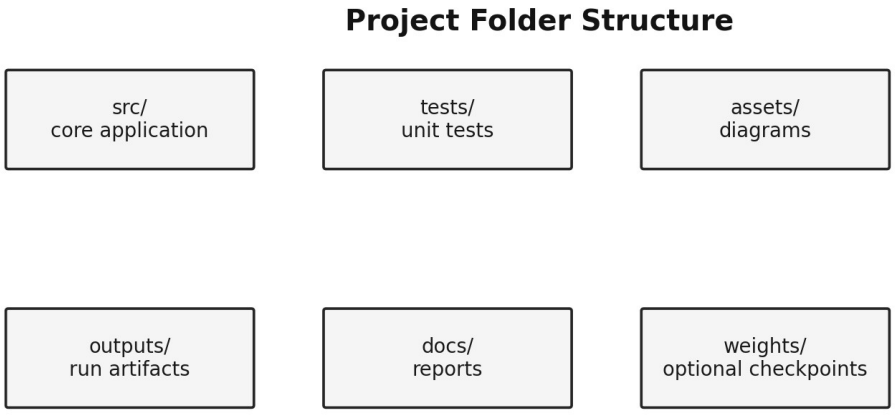


Figure 4. Project folder structure.

The source package contains separate modules for configuration, detection, I/O, metrics, overlays, and reporting, plus a small automated test suite.